

Interactive Discovery and Visualization of Process Trees

Wouter van der Heijden*

Stef van den Elzen†

Eindhoven University of Technology

ABSTRACT

Process Discovery, a key area of Process Mining, constructs models from event logs containing sequences of activity executions. Current automatic approaches suffer from opaque algorithms, limited incorporation of domain knowledge, and static, hard-to-explore visualizations. To address these limitations, we propose an interactive discovery process involving users in constructing process trees. This allows real-time interaction, dynamic domain expertise integration, and better understanding of the relationship between logs and models. We demonstrate this with a prototype featuring four linked views, and introduce improved process tree visualizations.

Index Terms: Visual Analytics, Process Mining, Process Trees.

1 INTRODUCTION

Many companies log interactions with systems to understand how processes execute and whether they conform to expectations. *Process Mining* (PM) focuses on automatically constructing representative models from *event logs* (e.g., [1, 6, 7, 8, 9, 10]), which record sequences of executed events (traces). Discovering process models using algorithms is known as *process discovery* [19, 20]. Discovered process models are typically large and complex, and there is no way for users to incorporate domain knowledge in the discovery process. With domain knowledge it potentially leads to smaller, less complex, and better-understood models. With automatic discovery, it is hard to know how an algorithm arrived at the resulting model. The lack of transparency coupled with the absence of domain knowledge, hinders understanding of the link between the event log and the resulting model. Finally, model visualizations are generally static, offering limited options for explorations and analysis [18]. In summary, complexity and explainability is challenging within PM. Current discovery implementations [7, 9] aim to remedy this by enabling users to set parameters before the algorithm execution, select a subset of traces, or manually edit models. However, key issues persist: 1) It is unclear how the model is obtained; 2) Algorithms neglect domain knowledge to construct models; 3) Users have limited ability to influence the discovery process; and 4) Visualizations can be hard to interpret and do not explicitly show the relation between event logs and discovered models. To tackle this, we aim to open up the black box of process discovery algorithms by exploring an interactive discovery process. This requires an algorithm that can adapt dynamically during execution. We leverage inductive mining [2, 3, 9, 17] to construct process trees, as its recursive, step-by-step structure naturally supports such interactivity.

2 BACKGROUND AND RELATED WORK

Relations in an event log can be abstracted into a *Directly Follows Graph* (DFG) modeling the direct succession between event pairs. An alternative data abstraction guaranteeing a sound process model is the process tree, obtained through inductive mining algorithms.

*e-mail: w.b.v.d.heijden@student.tue.nl

†e-mail: s.j.v.d.elzen@tue.nl

Inductive Mining — Inductive mining considers the directly follows relations between events and constructs a graph that is iteratively split into smaller subgraphs, until it consists of single activities. The properties these cuts correspond to are displayed in a *process tree* visualization. Inductive mining consists of many variations [9, 3, 17] differing in which constraints for the process trees are supported, whether infrequent or incomplete behaviour is taken into account, whether cuts are made in the log or on the DFG directly, and whether activity atomicity is considered. Figure 1 shows how a process tree is obtained using inductive mining.

Process Trees — A *process tree* is a tree-structured graph where leaf nodes represent individual activities, and internal nodes set restraints for a particular subset of activities. Each parent node governs its subtree by setting one constraint, visually represented by symbols: sequence ($\Rightarrow$) dictates that its branches must be executed in order from left to right. Exclusive or ($\times$) only allows a single branch to be executed. Concurrency ($\wedge$) allows each branch to be executed in any order; a branch does not have to be fully executed for another to start or be continued. Loop ($\odot$) nodes must execute their leftmost branch first, optionally followed by any other branch, followed by another leftmost branch execution. This can be done indefinitely. Interleaved ($\leftrightarrow$) is identical to concurrency, except branches must finish before others are started. Inclusive or ($\vee$) allows any subset of branches to be executed as long as at least one is executed. Figure 1d shows an example process tree. Research focused on process tree visualization is limited. A recent master thesis [23] addresses displaying additional performance statistics on process trees using color, but does not focus on visualizing process trees to explain its creation. The PRoM Framework [22] implements a large variety of plugins for process discovery. Inductive mining is supported through interaction by means of letting users set parameters for the underlying algorithms and for displaying the obtained model. Only one algorithm can typically be used to generate a model, and users cannot see how the model is derived. Beyond setting parameters, there is little opportunity to apply domain knowledge during construction. Cortado [15, 13] offers an interactive environment for building process trees, allowing users to start with an initial model and incrementally add traces. However, the creation of the initial model is not visualized, and when new behavior is introduced, the resulting process tree can change significantly, making it difficult to understand what changed and why. In summary, we observe:

- PM tree-based models are visualized using standard tree visualizations which are, generally, hard to interpret. Hierarchy-based visualizations are left mostly unexplored [23].
- Visualizations for event logs [24], do not explicitly visualize the relation to the model.
- Interaction techniques for inductive mining currently mostly consist of manually editing a visualization directly [14], or adapting visual parameters [14, 22].
- The application of domain knowledge has been explored by a select few [5, 15]. However, no approach enables users to apply domain knowledge *during* the discovery process.
- Current approaches are not transparent on how a model was obtained. The underlying algorithm(s) in current implementations remain a *black box*.

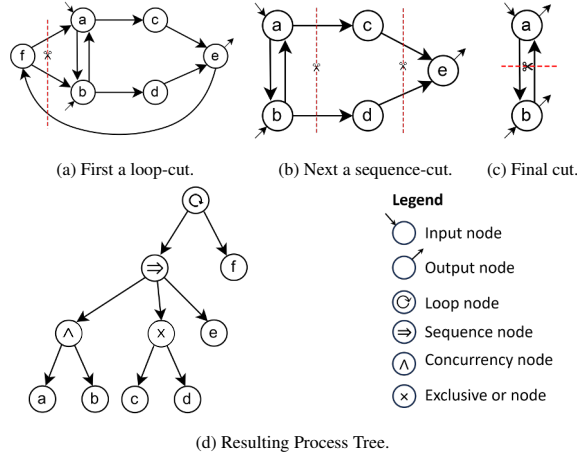


Figure 1: An example DFG (a) displaying the different cuts involved (a-c) to construct the process tree (d).

3 USERS AND WORKFLOW TASKS

Our approach targets three main user groups: (1) developers of process tree models seeking greater explainability and incorporation of domain knowledge; (2) educators who benefit from visual, step-by-step construction to teach process mining concepts; and (3) researchers and PM developers aiming to explore and improve the discovery process itself.

Workflow — Unlike traditional workflows, where users can only modify inputs or post-process the final model, our method makes the inductive mining algorithm interactive. Users are involved throughout model construction by selecting from algorithm-generated options or manually defining splits. This provides transparency, integrates domain knowledge, and maintains model interpretability by ensuring minimal changes to the tree at each step.

To further illustrate how our approach differs from the current workflow, we identify tasks and the flow between these in Figure 2. Users iteratively guide the algorithm by choosing how to split the event log at each stage. After each split, the corresponding DFG is generated to visualize trace relations and highlight available cut options. When a cut is applied, the process tree is updated, and the next branch is explored with a new DFG and cut set. This continues until all traces are processed. Users can also revise previous decisions by selecting tree nodes and redefining cuts. Changes in any view (traces, tree, DFG, or cuts) propagate across all others, supporting consistent, interactive exploration. The system also allows for direct manipulation of tree elements and visual parameters (e.g., encodings and layout), enhancing analysis and usability.

4 INTERACTIVE DISCOVERY AND VISUALIZATION

In alignment with the identified user tasks, the proposed interactive discovery and visualization of process trees consists of four perspectives, the *trace view*, the *process tree visualization*, the *DFG view*, and the *cuts view*. Figure 3 illustrates the layout of the different coordinated components as implemented in our prototype application [21]. Below, we further describe and discuss each component’s functionality, visualization, and, user interaction.

Trace View — The events are presented as a sequence of colored blocks, with colors consistent over all views for quick association. Trace variants are ordered by frequency and can be collapsed and expanded for a focused analysis. Users are enabled to study the set of traces for a selected process tree node. Additional statistics are shown to quickly compare different sets of traces.

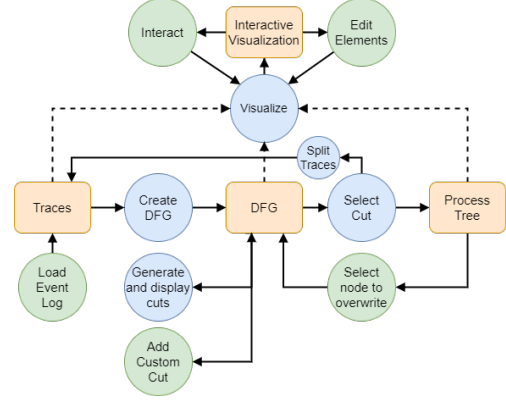


Figure 2: Proposed interactive process discovery approach. Green elements are user actions, blue elements are application actions.

Process Tree Visualization — In the Process Tree view, users inspect and analyze the model that has been constructed so far. The default process tree layout is generated using the Sugiyama layered tree layout [16]. Nodes can be selected and repositioned, to align to the user’s mental map for interpretability and analysis. Leaf nodes in both the process tree visualization and the DFG are colored according to the associated event. Labels are initially hidden to reduce clutter, but can be inspected in the Trace View, and hovering reveals additional information such as event names for leaf nodes, cut-types for inner nodes, and information about displayed icons. Important for the process tree analysis is the inspection of the involved trace frequencies. The total frequency of an event or set of traces can be, at most, equal to a certain local maximum. This local maximum is determined by the root node but can be overwritten by a loop node. If the frequency of any such event or set of traces is not equal to its local maximum (indicated with a red triangle icon, see Figure 3A), the execution of the node is optional, with the exception of (X) nodes. Optional icons indicate, e.g., badges to represent the number of loops observed in the data, and wrenches to indicate manual cuts.

Edge widths are used to convey frequencies and depend on the total frequency of the set of current traces belonging to the target node of the edge. Here, we chose a linearly scaled width over color encoding as this is a higher-ranked visual channel [12], and color is already reserved for event-type associations. When hovering over edges, exact frequencies are shown on demand. When an edge connects two identical node types, users are enabled to merge nodes (with the exception of loop nodes). This results in a single node as a replacement of the two nodes, where the children are combined into a single group while preserving order. This operation can be applied to the entire tree or manually for selected nodes. The merging of similar node types leads to a smaller, less complex, and better interpretable process tree model while preserving underlying semantics.

Custom Tree Layouts — The default layout typically used to draw process trees (e.g., Sugiyama [16]) does not take into account the underlying characteristics of the model, which might hinder readability and interpretability. One of the problems is that the default layout uses a fixed minimal distance between nodes, both vertically and horizontally. Their values are set to be as compact as possible while preventing overlap. This results in a clean layout where nodes are positioned in a grid-like fashion but leaves little room for flexibility and affects readability. Using varying distances enables us to position nodes to better represent the underlying traces. Also, current tree layouts use a layered hierarchical approach where child nodes are laid out in a top-to-bottom, left-to-right manner. As a result, the leaf nodes are no longer visually representing the correct order of the associated traces. See, for example, Figure 3A where

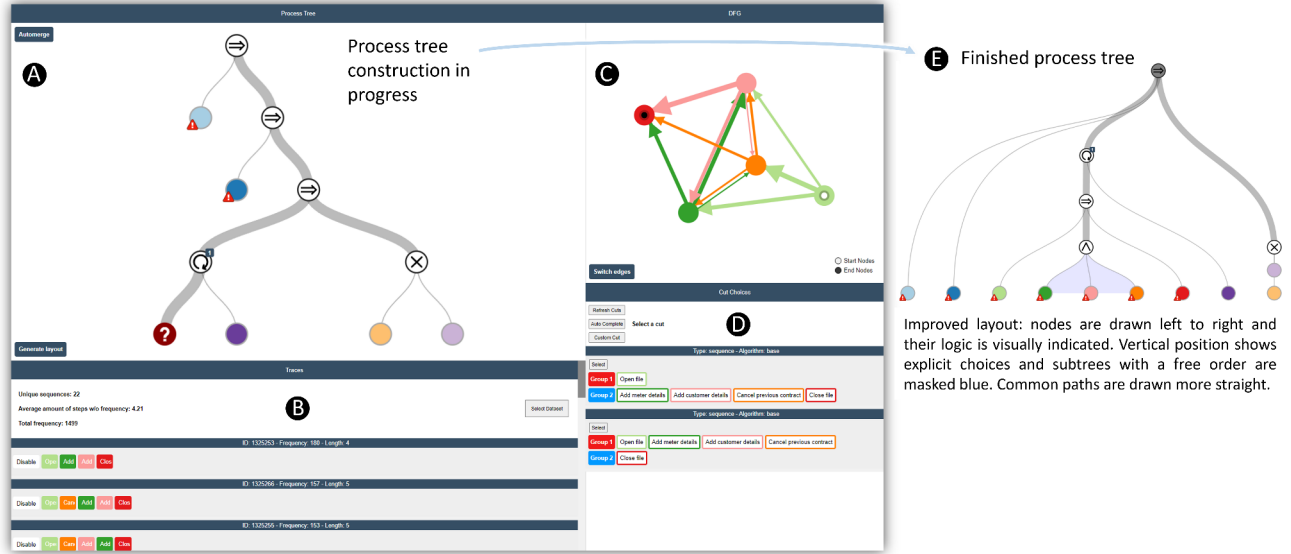


Figure 3: Interactive process tree discovery and visualization approach with four linked components; A) *Process Tree Visualization* B) *Trace View* - where users inspect the current set of traces associated with a process tree node. C) *DFG view* - an abstraction of all relations present in the current set of traces. D) *Cuts View* - to inspect and select cuts generated by the inductive mining algorithms, or manually define a cut.

the red (question-mark) event occurs before the blue events when read from left to right. In reality, the two blue events happen first due to the parent sequence operator. Furthermore, due to the layered approach, concurrent events appear in a sequence rather than occurring simultaneously. Also, loop operators inherit the above-described issues, which make them hard to interpret and link to the associated traces. Our aim is to have a custom tree layout where the sequence of events can be read from left to right. This way, the tree can be read comparably to how one would normally read and interpret traces. We make the following improvements (see Figure 5 for a quick overview of the results):

(1) *Concurrency* node branches can be executed in any order. A branch does not have to be completed before starting a different branch. To indicate this logic, concurrency nodes are drawn with a light blue mask for the surface of their entire subtree (see Figure 3E). This mask is transparent, to support nested concurrency nodes (see e.g., Figure 4-Branch 3).

(2) *XOR* nodes and their children are positioned different to reflect their logic. Here, we use vertical position to demonstrate that only a single event of the XOR node's children can be executed. If an XOR node exclusively has single events as children, they are stacked on top of each other to represent a choice has to be made. If any child of the XOR node is not an event node, its entire subtree is raised vertically instead. These subtrees are stacked based on their width, causing a pyramid-like structure to emerge (see Figure 4). This re-ordering is possible as there is no inherent order in contrast to e.g., a sequence node. The use of vertical position also demonstrates the logic when reading the tree from left to right.

(3) *Frequency weighted parent position*; in contrast to default layouts, the placement of parent nodes is not the average x-value of its children, but is weighted to child importance based on edge frequencies. This way, common paths in the process tree are aligned as straight as possible.

Building the tree top to bottom with a default layout incurs many errors when attempting to align all remaining event nodes in a horizontal line after positioning inner nodes. Therefore, we create the layout bottom-up, such that placement errors can be repaired as they occur. As an added bonus, the tree is now better connected to the

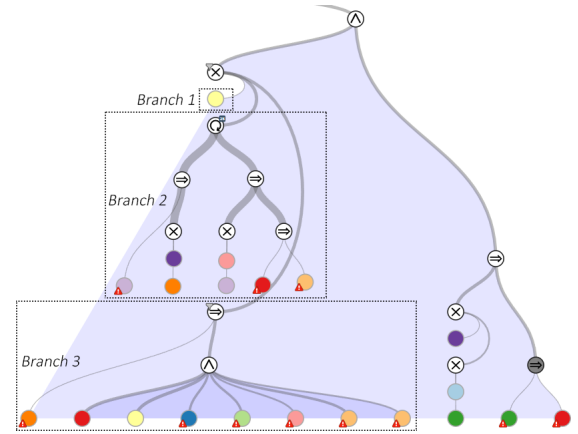


Figure 4: Multiple XOR subtrees stacked on top of each other to indicate their logic. Subbranches of the XOR node (3, 2 and 1) are sorted in order from small to wide, top to bottom.

underlying trace data. Furthermore, we enable users to manually reposition nodes as they see fit (e.g., based on domain knowledge).

The DFG View — The goal of the DFG view is two-fold. First, to quickly identify direct relations in the current set of traces, supported by the node colors (see Figure 3C). Start (white) and end (black) nodes are indicated by additional colored circles inside the nodes. Outgoing arrows inherit node color, enabling easy identification of the edge origin. Additionally, edge widths indicate frequencies of the relations between two events, to quickly identify important relations in the current set of traces. Finally, two edges are drawn in parallel if the relation is bidirectional. Second, the DFG displays possible cuts, guiding the model creation. As our work is focused on the interactive construction and visualization of the process tree, we opted for a force-directed layout of the DFG, which has the drawback of becoming hard to read when dealing with highly interconnected graphs. This is partly counter-acted by

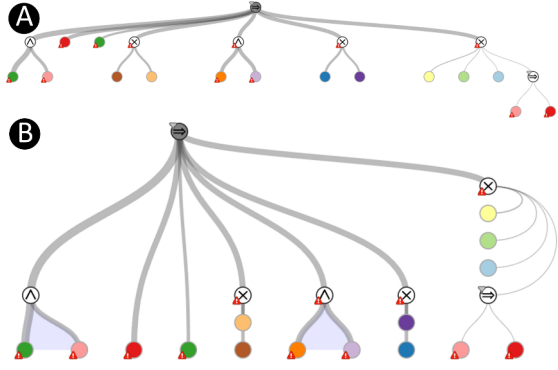


Figure 5: Default layout (A) and improved layout for readability (B) for the 0000104 dataset from PDC 2022.

enabling users to reposition nodes manually. However, better solutions exist (see *e.g.*, [11]), but this is left for future work.

The Cuts View — (see Figure 3-D) enables users to inspect suggested cuts generated by the implemented algorithms. This view is core to our approach as it enables users to apply their domain knowledge during the algorithm execution. It provides insight into the algorithm decisions and allows to deviate from this. Furthermore, it helps in connecting the data to the model, improving users’ ability to reason over model creation and structure. Each suggested cut displays its *type* (corresponding to a node type of a process tree), the *algorithm* used, and optional algorithm parameters. By selecting a suggested cut, the associated DFG is updated to reflect the cut by using color to represent the involved node groups (temporary changing the color of the nodes). The edges that are cut to form the two groups are colored differently. This enables users to understand the groups and the effect of the cut. Next to the suggested cuts, users can define custom cuts manually to apply domain knowledge in the construction. Furthermore, it enables users to construct a process model that is more in line with the desired process rather than the one present in the data. A manual cut can cause parts of the tree to become inaccurate as none of the guarantees provided by algorithms are retained. For this reason, an icon is added to such nodes for easy identification. Finally, we enable users to *auto-complete* (parts of) the process tree by automatically greedily selecting suggestions iteratively until the (sub)tree is fully constructed. Afterward, users are free to make changes by *e.g.*, editing subbranches.

Scalability — The prototype handles large logs, we tested up to 1.5 million event occurrences. The main bottleneck is converting .xes files to a front-end-friendly JSON format, which is cached after initial processing. To improve performance, only the most frequent traces are shown and others are loaded on demand. For visual scalability, traces are compactly drawn and horizontally scrollable if needed. Dense DFGs are made navigable with interactive node dragging, and a custom layout algorithm reduces edge overlap to maintain readability. Merging similar node types also keeps process trees manageable, though large trees may still require subtree collapsing (left for future work). Event labels are color-coded by frequency to ensure more prominent events are visually distinct, though distinguishable colors are limited to around 20.

5 ROAD TRAFFIC FINE USE CASE

To demonstrate our approach, we use the publicly available *Road Traffic Fine Management Process* dataset [4], which records the handling of traffic fines in an Italian municipality. The dataset contains over 500k events and represents an administrative process with substantial behavioral variety and complexity. We selected this real-world dataset because it presents a typical scenario for process discovery:

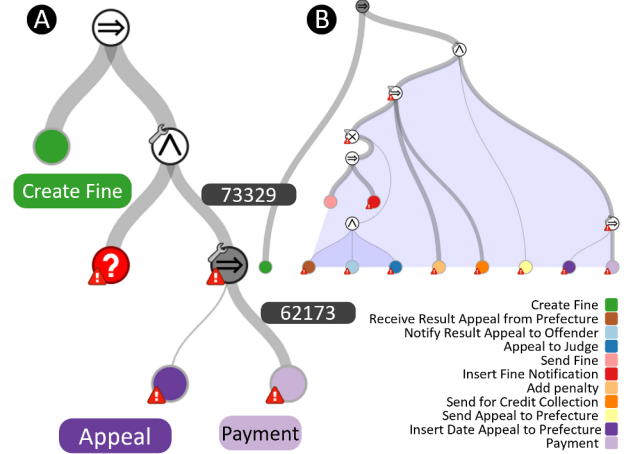


Figure 6: Interactive discovery of the Road Traffic Fine Management Process dataset [4]. After create fine, a split is made based on appeals, and shown how many eventually lead to payment (A). The final process tree constructed through synergy between automatic and user defined steps is shown in (B).

covery: the process includes conditional branching, repetition, and a large number of trace variants. This makes it well-suited to highlight the benefits of interactive, explainable model construction.

We begin by inspecting the initial DFG where the system proposes possible splits. The first split is a *sequence* of ‘Create Fine’ (left branch) followed by all other events (right branch). The system then suggests to split the right branch with a *concurrency* of all other events (left) and a ‘Send Appeal to Prefecture’. This is suggested because this event occurs in many different places in the traces. While useful, we are interested in how different payment behaviors affect the process structure. Therefore, we decide to split the process based on whether a fine was appealed, and further split the appeal cases based on whether payment was eventually received. We quickly see that from the 73329 cases that are appealed, 62173 lead to an eventual payment (see Figure 6A). By further construction we discover that cases without payment often terminate prematurely, while paid appeals lead to a secondary payment step. At each step, we explore and adjust the resulting tree structure, visual encodings, or revisit previous choices, ensuring the process tree remains explainable and consistent. The final tree is shown in Figure 6B. This transparent interactive discovery facilitates validation, especially for domain experts who often need to communicate the process logic further.

6 CONCLUSION

In this paper we aimed to transform the current workflow of process tree model discovery by more closely involving users in the creation process. This enables insights into the model creation process, as well as the model itself, ultimately leading to a better understanding of both. Furthermore, this provides users with a means to incorporate domain knowledge during the discovery phase. As a result, this potentially leads to smaller, less complex, and better interpretable process tree models. Furthermore, we focused on improving the tree visualization to better represent the underlying traces and easier interpretation of the inner nodes involved, by adapting the layout in such a way that it can be read from left to right and directly corresponds to the traces.

For future work, it is interesting to explore if the interactive approach also extends to other PM discovery algorithms and validate the approach through user studies.

ACKNOWLEDGMENTS

We gratefully acknowledge Dirk Fahland, Associate Professor in Process Analytics on Multi-Dimensional Event Data of the Analytics for Information Systems group at Eindhoven University of Technology, for thoughtful input and stimulating discussions during the early stages of this research.

REFERENCES

- [1] W. Aalst, V. Rubin, H. Verbeek, B. Dongen, E. Kindler, and C. Günther. Process mining: A two-step approach to balance between underfitting and overfitting. *Software and Systems Modeling*, 9:87–111, 11 2010. doi: 10.1007/s10270-008-0106-z 1
- [2] D. Brons, R. Scheepens, and D. Fahland. Striking a new balance in accuracy and simplicity with the probabilistic inductive miner, 2021. doi: 10.1109/ICPM53251.2021.9576864 1
- [3] J. Buijs, B. van Dongen, and W. van der Aalst. A genetic algorithm for discovering process trees. In *2012 IEEE Congress on Evolutionary Computation*, pp. 1–8, 2012. doi: 10.1109/CEC.2012.6256458 1
- [4] M. M. de Leoni and F. Mannhardt. Road traffic fine management process. https://data.4tu.nl/articles/dataset/Road_Traffic_Fine_Management_Process/12683249/1, 2015. doi: 10.4121/uuid:270fd440-1057-4fb9-89a9-b699b47990f5 4
- [5] P. M. Dixit, J. C. A. M. Buijs, W. M. P. van der Aalst, B. F. A. Hompes, and J. Buurman. Using domain knowledge to enhance process mining results. In P. Ceravolo and S. Rinderle-Ma, eds., *Data-Driven Process Discovery and Analysis*, pp. 76–104. Springer International Publishing, Cham, 2017. doi: 10.1007/978-3-319-53435-0_4 1
- [6] A. A. Freitas. *A Survey of Evolutionary Algorithms for Data Mining and Knowledge Discovery*, pp. 819–845. Springer Berlin Heidelberg, Berlin, Heidelberg, 2003. doi: 10.1007/978-3-642-18965-4 1
- [7] M. Ghafari and C. Tjortjis. A survey on association rules mining using heuristics. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 9:e1307, 04 2019. doi: 10.1002/widm.1307 1
- [8] T.-P. Hong, C.-H. Chen, and J. C.-W. Lin. *A Survey of Fuzzy Data Mining Techniques*, pp. 329–354. Springer International Publishing, Cham, 2016. doi: 10.1007/978-3-319-39014-7 1
- [9] S. J. J. Leemans. *Robust Process Mining with Guarantees - Process Discovery, Conformance Checking and Enhancement*, vol. 440 of *Lecture Notes in Business Information Processing*. Springer, 2022. doi: 10.1007/978-3-030-96655-3 1
- [10] A. Medeiros, B. Dongen, W. Aalst, and A. Weijters. Process mining for ubiquitous mobile systems: An overview and a concrete algorithm. vol. 3272, pp. 151–165, 06 2004. doi: 10.1007/978-3-540-30188-2 1
- [11] R. J. Mennens, R. Scheepens, and M. A. Westenberg. A stable graph layout algorithm for processes. *Computer Graphics Forum*, 38(3):725–737, 2019. doi: 10.1111/cgf.13723 4
- [12] T. Munzner. *Visualization Analysis and Design*. A K Peters / CRC Press, 2014. doi: 10.1201/b17511 2
- [13] D. Schuster, N. Focking, S. J. van Zelst, and W. M. P. van der Aalst. Incremental discovery of process models using trace fragments. In C. Di Francescomarino, A. Burattin, C. Janiesch, and S. Sadiq, eds., *Business Process Management*, pp. 55–73. Springer Nature Switzerland, Cham, 2023. doi: 10.1007/978-3-031-41620-0_4 1
- [14] D. Schuster, S. J. van Zelst, and W. M. van der Aalst. Cortado: A dedicated process mining tool for interactive process discovery. *SoftwareX*, 22:101373, 2023. doi: 10.1016/j.softx.2023.101373 1
- [15] D. Schuster, S. J. van Zelst, and W. M. P. van der Aalst. Incremental discovery of hierarchical process models. In F. Dalpiaz, J. Zdravkovic, and P. Loucopoulos, eds., *Research Challenges in Information Science*, pp. 417–433. Springer International Publishing, Cham, 2020. doi: 10.1007/978-3-030-50316-1_25 1
- [16] K. Sugiyama, S. Tagawa, and M. Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi: 10.1109/TSMC.1981.4308636 2
- [17] N. Tax, N. Sidorova, R. Haakma, and W. M. van der Aalst. Mining local process models. *Journal of Innovation in Digital Ecosystems*, 3(2):183–196, 2016. doi: 10.1016/j.jides.2016.11.001 1
- [18] S. van den Elzen, M. Jans, N. Martin, F. Pieters, C. Tominski, M.-C. Villa-Uriol, and S. J. van Zelst. Towards multi-faceted visual process analytics. *Information Systems*, 133:102560, 2025. doi: 10.1016/j.is.2025.102560 1
- [19] W. van der Aalst. Process discovery: Capturing the invisible. *IEEE Computational Intelligence Magazine*, 5(1):28–41, 2010. doi: 10.1109/MCI.2009.935307 1
- [20] W. van der Aalst and J. Carmona. *Process Mining Handbook*. Lecture Notes in Business Information Processing. Springer International Publishing, 2022. doi: 10.1007/978-3-031-08848-3 1
- [21] W. van der Heijden and S. van den Elzen. Github: Interactive discovery and visualization of process trees prototype. <https://github.com/stefvandenelzen/process-tree-builder>, 2025. 2
- [22] B. F. Van Dongen, A. K. A. de Medeiros, H. Verbeek, A. Weijters, and W. M. van Der Aalst. The prom framework: A new era in process mining tool support. In *Applications and Theory of Petri Nets 2005: 26th International Conference, ICATPN 2005, Miami, USA, June 20-25, 2005. Proceedings 26*, pp. 444–454. Springer, 2005. doi: 10.1007/11494744_25 1
- [23] R. Wolffensperger. *Static and Dynamic Visualization of Quality and Performance Dimensions on Process Trees*. Msc thesis, Mathematics and Computer Science, 2015. MSc Thesis. 1
- [24] A. Yeshchenko and J. Mendling. A survey of approaches for event sequence analysis and visualization. *Information Systems*, 120, 2024. doi: 10.1016/j.is.2023.102283 1